

Sistema de Gerenciamento e Planejamento de Missões Espaciais

Aluno: Leandro Castro Delevati

Linguagem: Python

1. Fonte de Dados

Item	Descrição
Nome da API	The Solar System OpenData
Endereço base	https://api.le-systeme-solaire.net/rest/bodies
Autenticação	Bearer Token, lido da variável de ambiente SOLAR_API_KEY
Formato de retorno	JSON, lista de objetos sob a chave "bodies"
Data de consulta	30/09/2026

Justificativa da escolha:

A API oferece uma quantidade grande e diversa de corpos celestes, cada um com atributos físicos e orbitais suficientes para dar suporte às operações de busca, pesquisa, filtragem e ao algoritmo guloso de planejamento de missão.

Endpoint utilizado:

```
GET /rest/bodies
Headers: Authorization: Bearer <SOLAR_API_KEY>
```

Exemplo de um item retornado:

```
{
  "id": "terre",
  "englishName": "Earth",
  "bodyType": "Planet",
  "isPlanet": true,
  "semimajorAxis": 149598023,
  "mass": { "massValue": 5.9722, "massExponent": 24 },
  "gravity": 9.8,
  "meanRadius": 6371.0,
  "avgTemp": 288,
  "moons": [ { "moon": "Moon", "rel": "..." } ],
  "discoveredBy": "",
  "discoveryDate": ""
}
```

- Faz GET real na API em tempo de execução, usando requests, e converte o JSON retornado em objetos CorpoCeleste. (Também criei uma opção para ler diretamente de arquivo local)
- **pip install -r requirements.txt** (Dependência para fazer requisições)
- Você precisa da própria chave API se quiser fazer requests:
 - `export SOLAR_API_KEY='seu-token-aqui'` (Bash)
 - `$env:SOLAR_API_KEY = "seu-token-aqui"` (Powershell)

2. Modelagem

Entidade: CorpoCeleste

Local: src/models/corpo_celeste.py

O papel desta classe é isolar o restante do sistema do formato bruto da API, se a API mudar um nome de campo só este arquivo precisa ser ajustado. A conversão é centralizada em `api_dict`, tanto para o retorno da API quanto para o arquivo `.json` local, os dois níveis de aquisição convergem para o mesmo formato interno.

Atributo	Tipo	Origem no JSON	Descrição
id	str	id	Identificador único usado como chave na Tabela Hash
nome	str	englishName	Nome em inglês, usado nas pesquisas
tipo	str	bodyType	Star, Planet, Dwarf Planet, Asteroid, Comet, Moon
e_planeta	bool	isPlanet	Flag booleana
distancia_km	float	semimajorAxis	Distância média ao corpo central
massa_kg	float	mass.massValue * 10 ^{massExponent}	Já calculada no <code>api_dict</code>
gravidade	float	gravity	Gravidade superficial (m/s ²)
raio_medio_km	float	meanRadius	Raio médio
temperatura_media_k	float	avgTemp	Temperatura média (K)
quantidade_luas	int	len(moons)	0 se o campo não existir
descoberto_por	Optional[str]	discoveredBy	Opcional
data_descoberta	Optional[str]	discoveryDate	Opcional

3. Estrutura de dados

Interface: IEstruturaBusca

Local: src/estruturas/[interfaces.py](#)

IEstruturaBusca é uma classe abstrata que define o contrato mínimo que **qualquer** estrutura deve seguir, independentemente do seu funcionamento interno. Isso significa que, na Parte 2, ao implementar Trie e Árvore B, bastará fazê-las herdar de IEstruturaBusca, assim como a Tabela Hash já herda, e implementar os mesmos seis métodos. O restante do sistema não precisará ser alterado.

Métodos declarados:

Todos os métodos são marcados com `@abstractmethod`, o que impede a instanciação de qualquer subclasse que não os implemente todos, Python levanta `TypeError` em tempo de execução se isso for tentado.

Método	Assinatura	Papel
inserir	(chave: str, valor: Any) -> None	Insere ou atualiza o valor associado a uma chave
buscar	(chave: str) -> Optional[Any]	Retorna o valor associado à chave, ou None se não existir
remover	(chave: str) -> bool	Remove a chave; retorna True/False indicando se ela existia
existe	(chave: str) -> bool	Verifica presença de uma chave, sem retornar o valor
todos_valores	() -> List[Any]	Retorna todos os valores armazenados, usado pelas operações de listagem/pesquisa/filtragem (Seção 6.2 e 6.3)
__len__	() -> int	Permite <code>len(estrutura)</code> , usado por <code>SistemaMissoes.quantidade_corpos()</code>

Estrutura implementada: Tabela Hash

Local: `src/estruturas/tabela_hash.py`

Justificativa da escolha: Escolhi implementar a tabela hash na parte 1 pois é a estrutura que estou mais familiarizado. A Tabela Hash resolve a busca por ID em tempo médio $O(1)$, o que a Trie (otimizada para prefixo) e a Árvore B (otimizada para ordenação/faixa) não fazem tão bem.

Implementação:

- Encadeamento externo: cada posição do vetor de buckets é uma lista de pares (chave, valor), nenhuma estrutura nativa de alto nível é usada para o armazenamento chave-valor.
- Função de hash própria, aplicando módulo a cada passo para evitar overflow:

```
h = 0
for caractere in chave:
    h = (h * 31 + ord(caractere)) % capacidade
```

- Redimensionamento dinâmico: quando o fator de carga ultrapassa 0.7, a capacidade dobra e todos os elementos são reinseridos (_redimensionar).

Instrumentação obrigatória:

Exposta ao usuário pela opção 9 do menu (imprimir_estatisticas_hash), que imprime capacidade atual, elementos armazenados, fator de carga, total de colisões e total de redimensionamentos.

Métrica	Onde é medida
total_colisoes	Incrementada em inserir() sempre que o bucket de destino já tem algum elemento, e também durante _redimensionar(), ao reposicionar os elementos na nova tabela
total_redimensionamentos	Incrementada a cada _redimensionar()
fator_carga	Propriedade calculada (quantidade_elementos / capacidade)

4. Funcionalidades do Sistema

Local: src/sistema.py

Todas as operações de consulta e filtragem são expostas ao usuário pelo menu em main.py (opções 3 a 9), que apenas coleta a entrada e delega para SistemaMissoes, nunca acessando TabelaHash diretamente.

Consulta de elementos:

Operação: buscar_por_id(id_corpo: str) -> Optional[CorpoCeleste] (opção 3 do menu).

Localiza um corpo celeste específico a partir do seu identificador único (ex: "terre", "mars"), delegando diretamente para TabelaHash.buscar(). É a única operação de consulta que usa a estrutura de dados avançada em busca exata, complexidade **O(1) em média**, já que a Tabela Hash foi escolhida justamente por resolver esse tipo de acesso eficientemente. No pior caso teórico (todas as chaves colidindo no mesmo bucket), a busca degrada para **O(n)**, mas o redimensionamento dinâmico (fator de carga máximo 0.7) mantém isso improvável na prática.

Pesquisa por atributos:

Operações:

- `pesquisar_por_nome_parcial(trecho: str) -> List[CorpoCeleste]`: retorna todos os corpos cujo nome começa com o trecho informado (case-insensitive).
- `pesquisar_por_tipo(tipo: str) -> List[CorpoCeleste]`: retorna todos os corpos de um `bodyType` específico (Planet, Moon, Dwarf Planet, Asteroid, Comet).

Diferente da consulta por ID, pesquisa por atributo não é uma operação que a Tabela Hash acelera, ela só otimiza busca exata pela chave usada na indexação (id), não por outros campos. Por isso, ambas as operações fazem **busca linear $O(n)$** sobre `_tabela.todos_valores()`, percorrendo todo o conteúdo armazenado. Essa limitação é conhecida e documentada: a Parte 2 do trabalho pretende usar a Trie para acelerar pesquisa por prefixo de nome, tornando essa busca mais eficiente que **$O(n)$** .

Listagem e filtragem:

Operações:

- `listar_todos() -> List[CorpoCeleste]`: retorna todos os corpos carregados, sem nenhum critério de filtro. **$O(n)$** .
- `filtrar_por_faixa_distancia(minimo_km, maximo_km) -> List[CorpoCeleste]`: retorna os corpos cuja distância ao Sol está dentro de uma faixa [minimo, maximo] informada pelo usuário. Também **$O(n)$** , pois exige inspecionar `distancia_km` de cada elemento.
- `listar_ordenado_por_distancia() -> List[CorpoCeleste]`: retorna todos os corpos ordenados crescentemente por distância ao Sol, usando `sorted()`. **$O(n \log n)$** .

Assim como a pesquisa por atributo, a filtragem por faixa de distância é uma operação que se beneficiaria de uma estrutura ordenada por natureza, é exatamente o caso de uso que motiva a escolha futura da Árvore B na Parte 2 (otimizada para buscas por faixa/intervalo), reduzindo esse filtro de **$O(n)$** para algo próximo de **$O(\log n + k)$** , onde k é o tamanho do resultado.

Operações adicionais:

Operação: `imprimir_estatisticas_hash()`

Expõe ao usuário a instrumentação obrigatória da Tabela Hash): capacidade atual, quantidade de elementos armazenados, fator de carga, total de colisões e total de redimensionamentos ocorridos desde o carregamento dos dados.

Resumo de complexidade das operações:

Operação	Usa a Tabela Hash?	Complexidade
Consulta por ID	Sim (buscar)	$O(1)$ médio
Pesquisa por nome/tipo	Não (busca linear)	$O(n)$
Listar todos	Não	$O(n)$
Filtrar por faixa de distância	Não (busca linear)	$O(n)$
Listar ordenado por distância	Não	$O(n \log n)$
Estatísticas da Hash	Sim (métricas internas)	$O(1)$

5. Análise de complexidade amortizada - Rehashing da Tabela Hash

O problema com o pior caso "ingênuo":

Isoladamente, uma chamada a `_redimensionar()` custa $O(n)$, pois todos os n elementos armazenados são percorridos e reinseridos na tabela nova. Se essa análise fosse aplicada ingenuamente a cada inserção, seria fácil concluir, de forma excessivamente pessimista, que toda inserção custa $O(n)$ no pior caso. Isso não reflete o comportamento real do sistema ao longo de uma sequência de n inserções.

Método contábil:

Cobra-se de cada operação `inserir()` um custo amortizado constante, maior que seu custo real médio, e o excedente é guardado como "crédito" para pagar os redimensionamentos futuros.

- Seja a capacidade sempre dobrada quando o fator de carga ultrapassa 0.7 (`_fator_carga_maximo`).
- Custo real de uma inserção sem colisão de redimensionamento: $O(1)$ (cálculo do hash + append no bucket).
- Cobrando 3 unidades de crédito por inserção:
 - 1 unidade paga a própria inserção;
 - 2 unidades ficam depositadas no elemento recém-inserido.
- Quando o redimensionamento ocorre, cada um dos n elementos que precisam ser reinseridos já acumulou 2 créditos, exatamente o suficiente para pagar seu próprio recálculo de hash e reinserção na tabela nova (custo real $O(1)$ por elemento reinserido).

Como nenhuma operação nunca fica com saldo de crédito negativo, o custo amortizado por inserção é $O(1)$, mesmo que o custo de uma operação isolada de redimensionamento seja $O(n)$.

6. Estratégia Gulosa - Planejamento e Triagem de Missões

Local: src/missao/planejador_rota.py

Partindo do Sol (posição 0 no eixo de distância), o sistema monta uma rota que maximiza o potencial científico total da missão, respeitando duas restrições simultâneas de recurso:

Restrição	Significado
combustível_milhões_km	Orçamento total de combustível, expresso como a distância (em milhões de km) que a nave consegue percorrer. É consumido ao longo da missão e não é reabastecido.
custo_parada_milhoes_km	Combustível fixo gasto a cada parada (por exemplo, para entrar em órbita), na mesma unidade. O valor padrão é 0.

A nave nunca retorna a uma posição anterior (viagem estritamente progressiva, a favor da distância crescente ao Sol). Cada passo até um corpo consome:

$\text{custo_do_passo} = \text{distancia_do_trecho} + \text{custo_de_parada}$

Critério de seleção dos candidatos:

O benefício de cada corpo é o seu potencial científico, definido como o quanto sua densidade se desvia da densidade de referência (a da Terra, $\approx 5514 \text{ kg/m}^3$):

$\text{potencial_cientifico} = | \text{densidade_corpo} - 5514 |$

Corpos muito mais densos (núcleo metálico) ou muito menos densos (composição gelada/gasosa) que a Terra são tratados como mais interessantes para investigação. O custo de cada candidato é custo do trecho até ele + o custo da parada, a partir da posição atual da nave. A cada iteração, dentre os candidatos ainda não visitados, à frente da posição atual e alcançáveis com o combustível restante ($\text{custo_do_passo} \leq \text{combustivel_restante}$), o algoritmo escolhe o que maximiza a razão:

$\text{razão} = \text{potencial_cientifico} / \text{custo_do_passo}$

Corpos sem massa ou raio suficientes para calcular densidade são descartados dos candidatos (descartados_sem_dados). Luas também são excluídas, pois seu campo de

distância é relativo ao planeta que orbitam, não ao Sol, e misturá-lo na mesma régua geraria posições sem sentido físico.

Estratégia implementada:

```
posição_atual = 0 (Sol)
combustível_restante = combustível_milhoes_km

enquanto combustível_restante > 0:
    para cada candidato não visitado à frente de posição_atual:
        trecho = distância(candidato) - posição_atual
        custo = trecho + custo_de_parada
        se custo > combustível_restante: descarta (fora de alcance)
        senão: calcula razão = potencial(candidato) / custo
    se nenhum candidato alcançável: encerra a missão
    senão:
        escolhe o candidato de maior razão
        posição_atual = distância(candidato escolhido)
        combustível_restante = combustível_restante - custo
```

A cada iteração, o algoritmo faz uma escolha local e definitiva (não há backtracking): decide o próximo destino olhando somente para o benefício imediato do próximo passo, sem considerar o impacto dessa escolha nos passos futuros.

Exemplo de execução:

Com um conjunto pequeno (Mercúrio, Vênus, Terra, Marte, Ceres e Júpiter), combustível de 10.000 milhões de km e custo de parada de 1.000 milhões de km:

Passo	Posição da nave (milhões de km)	Escolhido	Trecho (milhões de km)	Passo (milhões de km)	Razão (potencial/km)
1	0	Ceres	413,7	1.413,7	0,00000242 (Júpiter ficou em 0,00000235 e Marte em 0,00000129)
2	413,7	Júpiter	364,8	1.364,8	0,00000307 (único candidato à frente)
3	778,5	nenhum	-	-	-

Rota final: Ceres -> Júpiter, potencial científico total de ≈ 7.605 , com 2.778,5 milhões de km de combustível gastos e 7.221,5 milhões de km de sobra. Mercúrio, Vênus e Terra ficam de fora por terem potencial científico baixo (a Terra tem potencial zero). Marte é ultrapassado: a partir do Sol, Ceres tinha razão maior, e como a nave nunca retorna, Marte deixa de ser alcançável.

Justificativa do critério:

A razão potencial/custo (em vez do potencial absoluto) evita que o algoritmo sempre prefira o corpo cientificamente "mais exótico" independentemente do custo. Ela avalia o benefício pelo combustível realmente gasto para alcançá-lo.

O custo fixo de parada é necessário para que o problema tenha sentido. Como a nave só avança, a soma dos trechos é sempre igual à posição da última parada, de modo que, sem esse custo, parar em corpos intermediários seria gratuito e o ótimo seria trivial (visitar todos os corpos dentro do alcance). Com o custo fixo, cada parada consome combustível, e o algoritmo passa a pular corpos de baixo potencial. Um custo de parada pequeno aproxima o comportamento do caso sem custo, um custo grande favorece poucas paradas em corpos de potencial alto.

A escolha de densidade como fator de potencial científico se justifica porque a maioria dos corpos da API tem massa e raio registrados, então a densidade é calculável para muitos corpos, o que cobre uma faixa muito maior de candidatos.

Limitações da estratégia gulosa:

A escolha estritamente gulosa não garante a rota de potencial científico total máximo (ótimo global), por alguns motivos:

- **Sem visão de futuro:** o algoritmo escolhe, a cada passo, a melhor razão imediata, sem considerar se essa escolha vai "fechar o caminho" para destinos que, somados, rendem mais. No exemplo acima, Marte foi ignorado porque Ceres tinha razão maior a partir do Sol. Visitar Marte, Ceres e Júpiter custaria apenas 3.000 milhões de km em paradas mais 778,5 de distância, bem dentro do orçamento, e renderia um potencial total maior que o da rota escolhida (≈ 9.185 contra ≈ 7.605).
- **Decisões irreversíveis:** uma vez escolhido um destino, não há como o algoritmo voltar atrás caso, poucos passos depois, perceba que a decisão anterior impediu uma rota melhor. É uma característica de qualquer algoritmo guloso.
- **Sensibilidade ao custo de parada:** a rota depende do valor escolhido para `custo_parada_milhoes_km`, que é um parâmetro da missão e não um dado dos corpos.

Encontrar a rota de potencial máximo garantido exigiria explorar todos os subconjuntos possíveis de paradas (ou usar programação dinâmica sobre a posição e o combustível restante), o que teria custo computacional muito maior. A abordagem gulosa resolve o problema em tempo proporcional a `paradas × número_de_candidatos`, ou seja, no pior caso $O(n^2)$.